Path Planning with Obstacles

Ryan Noghani
Department of Computer Science
University of California, Santa Barbara
ryannoghani@ucsb.edu

***Abstract*—This report covers path planning with square and circular obstacles, a which is a nonconvex optimization problem. My main 3 efforts were getting the solutions to be globally optimal, constraining the entire continuous trajectory from hitting obstacles, and extending the example in lecture to work for square shaped obstacles. NOTE: you can find all my projects including code in this repo: https://github.com/ryannoghani/Path-Planning-With-Obstacles-ECE-594T-Project**

I. Introduction

Chapter 4 of the class course notes [1] covered nonconvex optimization methods. One of the examples shown was "path planning with obstacle-avoidance constraints", where given a starting and ending point, a robot must find the shortest distance path while avoiding 2D circular obstacles. These 2D circles cause the feasible set to become nonconvex, making this a nonconvex optimization problem. The approach used to solve this problem was "Difference of Convex Functions", and "Convex Relaxation and Rounding" later in the chapter to find a good intial trajectory for the CVXPY convex optimization solver to start with. This allows for the solver to find a globally optimal trajectory.

I found this problem interesting and wanted to do a project about it. Since the example only works with circular obstacles, I was curious if I could use the methods shown in class to work for other obstacle shapes. I was too ambitious and did not get as far as I wanted to with this, but I was able to create an algorithm that can handle squares.

My project also fixes the issue of when the lines connecting trajectory points cross through obstacles. I do this using Semi Infinite Programming (SIP), which I will talk more about in the report.

II. Technical Approach

A. Path planning with square obstacles

The example in class [1] first does linear restrictions on the constraints dealing with the obstacles. These constraints are of form $r_i - \|x(k) - c_i\|_2 \leq 0$ for $i = 1, ..., n$, where r_i is the radius of the i_{th} circle, $x(k)$ is the k_{th} point of the trajectory, c_i is the center of the i_{th} circle, and n is the number of circles in the problem.

The linear restriction comes from the idea of Difference of Convex Functions. I.e., the first term and second term r_i and $\|x(k) - c_i\|_2$ are both convex functions with respect to $x(k)$. Suppose I have an initial guess for the trajectory, where the trajectory points are $a_1, ..., a_K$. We have a lower bound for both functions using the first order condition for convexity, but specifically we use the lower bound for the norm term. This looks like:
$\|x(k) - c_i\|_2 \geq \|a_k - c_i\|_2 + (\frac{a_k - c_i}{\|a_k - c_i\|_2})^T (x(k) - a_k)$. This is simply saying the hyperplane approximation of
$\|x(k) - c_i\|_2$ centered at a_k is a lower bound for $\|x(k) - c_i\|_2$. This implies that
$-\|x(k) - c_i\|_2 \leq -\|a_k - c_i\|_2 - (\frac{a_k - c_i}{\|a_k - c_i\|_2})^T (x(k) - a_k)$, which implies that
$r_i - \|x(k) - c_i\|_2 \leq r_i - \|a_k - c_i\|_2 - (\frac{a_k - c_i}{\|a_k - c_i\|_2})^T (x(k) - a_k)$. The convex restriction is to constrain this upper bound by ≤ 0, AKA: $r_i - \|a_k - c_i\|_2 - (\frac{a_k - c_i}{\|a_k - c_i\|_2})^T (x(k) - a_k) \leq 0$
In case the difference between $x(k)$ and a_k is confusing, we are taking our previous trajectory solution $a_1, ..., a_n$ and are solving a new optimization problem with linear hyperplane constraints about $a_1, ..., a_K$ with variables $x_1, ..., x_n$. At first, these trajectories may hit the obstacles, but every iteration improves until no obstacles are hit and the Sequential Quadratic Program converges.

My implementation changes the constraint so that the obstacles being avoided are now squares. This is done using the infinity norm in constraints rather than the 2 norm. This looks like:
$r_i - \|x(k) - c_i\|_\infty \leq 0$. However, one issue I encountered is computing the tangent hyperplane upper bound requires computing the gradient. However, the infinity norm is not differentiable [2]. At first I thought of approximating the infinity norm with a p norm, where p is very large but not infinity. However, I realized this wasn't necessary and decided to use subgradients instead. Thus I define

$$\nabla_{x(k)} \|x(k) - c_i\|_\infty = \begin{cases} (1,0)^T, & |x_1(k) - c_{i,1}| > |x_2(k) - c_{i,2}|, \\ (0,1)^T, & \text{otherwise.} \end{cases}$$

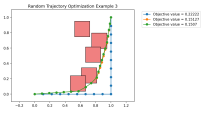
$x(k)$ and c_i are points on a 2D plane, so I am just referencing their components with subscripts above.

I deal with the gradient being undefined when
$|x_1(k) - c_{i,1}| = |x_2(k) - c_{i,2}|$ by breaking the tie and setting it to be [0, 1]. Implementing this gradient in code requires just an if else statement. The restricted, linearized constraint now looks like
$r_i - \|a_k - c_i\|_\infty - \nabla_{x_k} (\|a_k - c_i\|_\infty)^T (x(k) - a_k) \leq 0$, with $\nabla_{x(k)} (\|a_k - c_i\|_\infty)$ being $\nabla_{x(k)} \|x(k) - c_i\|_\infty$ evaluated at $x(k) = a_k$.

We are now able to choose points that avoid square obstacles. However, the figure below illustrates the following 2 issues:

1) The algorithm chooses a local minimum instead of global. I can see a better solution just by looking at the picture.
2) Even though the points don't go inside the squares, the lines that connect them do.

The next few sections will solve both of these issues.

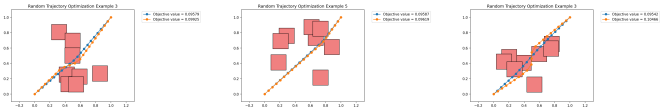


B. Global Minimum

The first issue I approached was getting my trajectories with square obstacles to be global rather than local minimums. Chapter 4 is able to achieve this with circle obstacles by relaxing the original nonconvex problem into a semi-definite program with variables $x(1), ..., x(k)$ and new variables $X(k) = [1, x(k), x(k+1)]^T[1, x(k), x(k+1)]$ [1]. The variables $x(1), ..., x(k)$ are then used as the initial local trajectory for the Difference of Convex Functions method in the last section. This increases the chances the Difference of Convex Functions method converges to a local optimum.

I was unsuccessful in finding a way to relax the square obstacle constraints. The relaxation method in [1] depends on being able to express the path planning with obstacles problem as a nonconvex QCQP [1]. What makes putting square obstacle constraints into QCQP form difficult is the infinity norm. The constraint is of the form $\|x\|_\infty \leq a$, which means at least one element of vector x must be $\geq a$ for the constraint to be satisfied. This condition resembles OR logic, where at least 1 constraint has to be satisfied for a point to be considered feasible, which is not the usual form of optimization problems. I am not sure if the QCQP to SDP relaxation is possible for square obstacles or not, so I opted for a temporary solution instead.

As a temporary solution, I add an extra relaxation by replacing the square obstacles with circles in the and do the usual SDP relaxation. I also considered using a high norm to approximate the infinity norm, but coming up with new P matrices for the SDP was difficult. I was surprised that making the squares circles in the relaxation worked well. Below is a figure that shows Difference of Convex Functions finding a global optimum with the addition of the SDP relaxation with circles.



The solutions are now considerably better now that they are finding global optimums rather than local. However, all 3 figures show the lines connecting the trajectory points still cross into the squares.

In order to solve this issue, I began looking at different research papers.

III. Literature Review

"Semi-infinite programming for trajectory optimization with non-convex obstacles" by Kris Hauser deals with enforcing collision constraints between discretized points on a trajectory [3]. Its solution is to use a semi-infinite-program (SDP), which is defined as the following in our problem's context:

$$\begin{aligned} \min_{x(1),\ldots,x(K-1)} \quad & \sum_{k=0}^{K-1} \|x(k+1) - x(k)\|_2^2 \qquad (1) \\ \text{s.t.} \quad & r_i - \|(1-t)x(k) + t\,x(k+1) - c_i\|_2 \leq 0, \\ & \forall i = 1, \ldots, n,\ k = 0, \ldots, K-1,\ t \in [0,1], \\ & x(0) = p_0, \\ & x(K) = p_f. \end{aligned}$$

The introduction here is the scalar real number t. Adding t allows us to check every point on the line connecting two adjacent points enforces the obstacle constraints. In theory, solving this optimization problem gives a trajectory where no points and their lines connecting it ever overlap with obstacles.

In general, directly solving SIP's can be difficult since optimization solvers can not handle the continuous variable t in the constraints. What [3] suggests instead is to find some t^* that maximizes the left hand side of its constraint. If we find t^*, we can determine whether it satisfies the constraint. If it doesn't, then the points $x(k)$ and $x(k+1)$ that form the line are infeasible, since there exists at least one t causing the constraint to fail. But if t^* does satisfy the constraint, all of the other constraints are satisfied by transitivity. To make this clear, I reformulate the SIP below into the following optimization problem:

$$\begin{aligned} \min_{x(1),\ldots,x(K-1)} \quad & \sum_{k=0}^{K-1} \|x(k+1) - x(k)\|_2^2 \qquad (2) \\ \text{s.t.} \quad & r_i - \|(1-t^*_{i,k})x(k) + t^*_{i,k}\,x(k+1) - c_i\|_2 \leq 0, \\ & \forall i = 1, \ldots, n,\ k = 0, \ldots, K-1, \\ & x(0) = p_0, \\ & x(K) = p_f. \end{aligned}$$

Note I turned t^* into $t^*_{i,k}$ to show it is a unique value for each obstacle constraint. For the sake of simplicity, I will now refer to t^* as the t that maximizes the left side of some arbitrary obstacle constraint. Since we choose t^* such that $r_i - \|(1-t)x(k) + t\,x(k+1) - c_i\|_2 \leq r_i - \|(1-t^*_{i,k})x(k) + t^*_{i,k}\,x(k+1) - c_i\|_2, \forall t \in [0,1]$, by transitivity enforcing the constraint $r_i - \|(1-t^*_{i,k})x(k) + t^*_{i,k}\,x(k+1) - c_i\|_2 \leq 0$ implies that $r_i - \|(1-t)x(k) + t\,x(k+1) - c_i\|_2 \leq 0, \forall t \in [0,1]$.

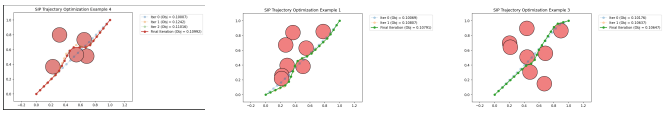
The question now is how we solve for t^*. In the next 2 sections, I will show how to solve for it when are obstacles are circles versus when they are squares.

A. t^* For Circular Obstacles

To make the math more simple, I am rewriting the obstacle constraints to the form $r_i \leq \|(1-t)x(k) + t\,x(k+1) - c_i\|_2$.

Now to make sure this constraint is satisfied for all t, we must find the t^* that minimizes the norm on the right side of the inequality.
$\|(1-t)x(k)+t\,x(k+1)-c_i\|_2$ is convex and differentiable with respect to t, so we know setting its derivative to 0 and getting solving for t gives us the global minimum. To simplify the math further we can instead take the equivalent constraint $r_i^2 \leq \|(1-t)x(k)+t\,x(k+1)-c_i\|_2^2$ and rewrite the right side as $\|a+bt\|_2^2$, where $a = x(k)-c_i$ and $b = x(k+1)-x(k)$. The derivative of $\|a+bt\|_2^2$ with respect to t is
$2a^Tb + 2t\,\|b\|_2^2$. The solution is
$t = \frac{-a^Tb}{\|b\|_2^2} = \frac{-(x(k)-c_i)^T(x(k+1)-x(k))}{\|x(k+1)-x(k)\|_2^2}$. However, this solution is not necessarily t^*, since t^* must in the interval $[0,1]$. Thus we project the solution to 0 if it is negative, and 1 if it is positive.

Integrating these new constraints into the Difference of Convex Functions restrictions, we successfully avoid obstacles in our entire trajectory. See the figures below.



B. t^* For Square Obstacles

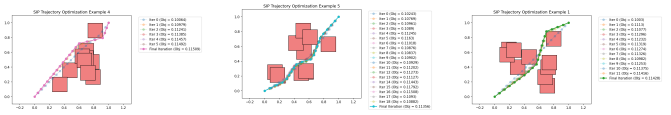
Because the infinity norm isn't differentiable, we instead solve for t^* another way. Let $a = x(k) - c_i$ and $b = x(k+1) - x(k)$. So we get
$\|(1-t)x(k)+t\cdot x(k+1)-c_i\|_\infty = \|a+bt\|_\infty$. a and b are 2 component vectors, so
$\|a+bt\|_\infty = max(|a_1+b_1t|, |a_2+b_2t|)$. Considering different cases with the absolute value requires some effort, but doing some calculations, the candidates that minimize the norm are the following:

$$t \in \left\{0, 1, -\frac{a_1}{b_1}, -\frac{a_2}{b_2}, \frac{a_2-a_1}{b_1-b_2}, -\frac{a_1+a_2}{b_1+b_2}\right\}.$$

Substituting a and b and plugging the candidate values t in, we get:

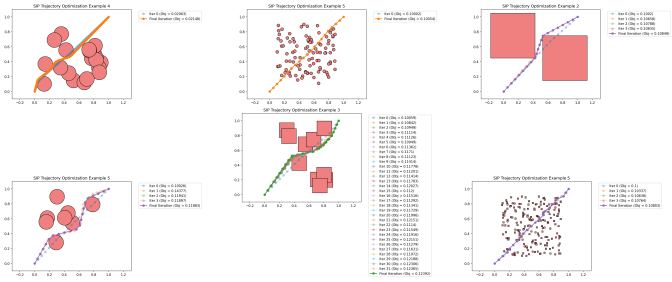
$$\begin{aligned} t^* = \min\Big\{ & \max(|a_1|, |a_2|), \\ & \max(|a_1+b_1|, |a_2+b_2|), \\ & \max\left(0, \left|a_2 - \frac{b_2}{b_1}a_1\right|\right), \\ & \max\left(\left|a_1 - \frac{b_1}{b_2}a_2\right|, 0\right), \\ & \max\left(\left|a_1 + \frac{a_2-a_1}{b_1-b_2}b_1\right|, \left|a_2 + \frac{a_2-a_1}{b_1-b_2}b_2\right|\right), \\ & \max\left(\left|a_1 + \frac{-a_1-a_2}{b_1+b_2}b_1\right|, \left|a_2 + \frac{-a_1-a_2}{b_1+b_2}b_2\right|\right)\Big\}. \end{aligned} \quad (3)$$

This looks confusing, so I added some notes inside my repository that show my work. But the result of doing this is shown in the figures below, where now the square obstacles are being completely avoided.



IV. Results

Below are some more examples of the algorithm:



V. Strengths/Limitation Discussion

A definite strength of the algorithm is it now is able to avoid obstacles completely. However, I believe there's still room for improvement for the actual solutions the algorithm finds. Take the bottom left figure in the results section for example. By just looking at the path, I am suspicious that this is truly the global optimum. Although I am not completely sure, I see a shorter path by going below all of the circles.

Another limitation of my algorithm is only being able to handle squares and circle obstacles. Extending this more generally to convex shapes might be difficult. Part of the reason I chose squares was because their constraint can simply be written in terms of an infinity norm, similar to how the circle constraints can be expressed with the 2 norm.

Also, as I increase the number of obstacles in the problem, I notice that the chance the optimization solver is able to find a solution becomes low. I am not sure if there is truly no feasible solution or because of a bug in my code, so this is something I want to investigate further.

VI. Potential Next Steps

1) Modify algorithm to handle a mix of circle and square obstacles.
2) Find better approach to integrate square obstacles into SDP relaxation than just approximating squares as circles.
3) Add more types of shapes for algorithm to handle.
4) Try a different method to solve (like GCS).
5) Further explore research in topic. I know I only have 1 reference in this report, which shows I did not use outside findings as much as I should have.

References

[1] Marcucci, T. (2026). Chapter 4 Continuous Nonconvex Optimization Lecture Notes. Optimization and Learning for Robotics and Control, University of California, Santa Barbara.

[2] H. H. Sohrab, *Basic Real Analysis*, 2nd ed. New York, NY, USA: Birkhäuser, 2014.

[3] K. Hauser, "Semi-infinite programming for trajectory optimization with non-convex obstacles," *International Journal of Robotics Research*, vol. 40, no. 10–11, pp. 1106–1122, 2021, doi: 10.1177/0278364920983353.